

Review

Rendering pipeline fundamentals

What is rendering pipeline:

A process consists of a series of stages that allows a 3D scene to be converted into a 2D image for the screen by passing in data, and the final product is an image.

Components of the pipeline:

Vertex array: data contains the positions in world space of the vertices

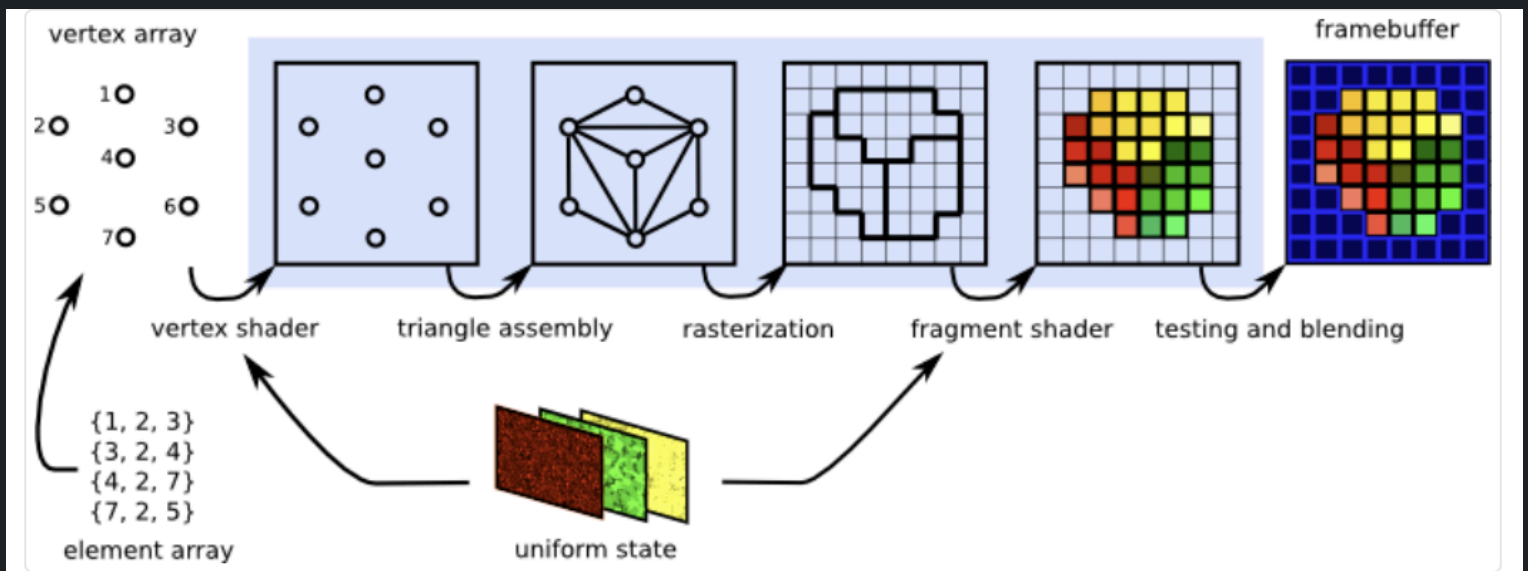
Vertex shader: Convert the input data from world space to screen space (camera's perspective)

Rasterization: Determine the pixels after triangle assembly, filling in to create the basic shape

Fragment shader: Determines the color of the pixels, can be used for more complex calculations such as lighting (ie: pixel is lit or dim).

Test/Blending: Depth test to check and determine whether the pixels are visible or not.

Typical diagram of the rendering pipeline:



Examples of what each

Examples seen in class:

Vertex shaders can let us manipulate the geometry of the meshes, for example making a wavy effect for water, wind effects on grass and flags. Fragment shader can let us recolor textures, or even the whole scene using color correction.

Deferred and forward rendering:

What are forward and deferred rendering:

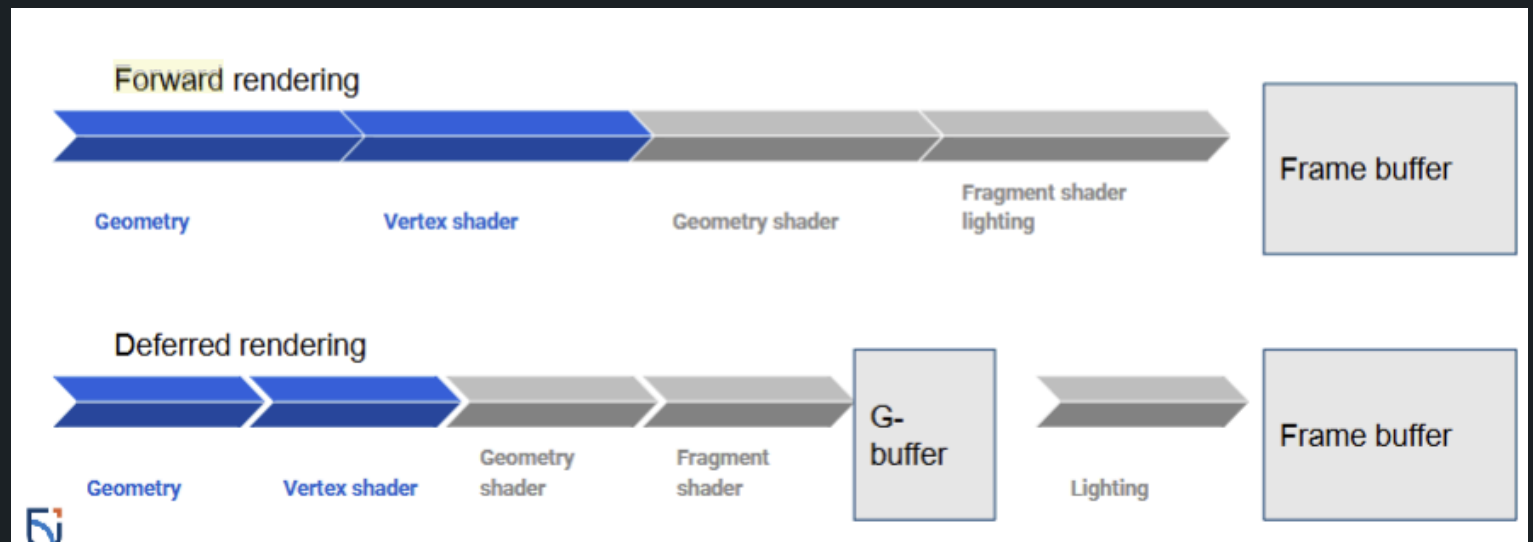
Two kinds of techniques for calculating the lighting and material properties. Forward rendering (classic approach) processes each object and applies lighting passes on it (simpler and faster, but costly due to numerous amounts of light). This is linear, passing each geometry down one at a time to produce the final image.

Deferred rendering has two stages: renders data into G-buffer (term means all textures used to store lighting data for the final pass: includes color, normals, depths), and calculates the lighting based on the buffer (more efficient). Wait a bit to combine all and apply shading at the end, this means many lights can be added, no limits to shadows, and doesn't cause additional passes. However, it can't handle transparency well

Applications:

Lighting calculations, can be used to stylize and enhance the image.

Diagram:



Vertex and fragment shaders

What are vertex and fragment shaders:

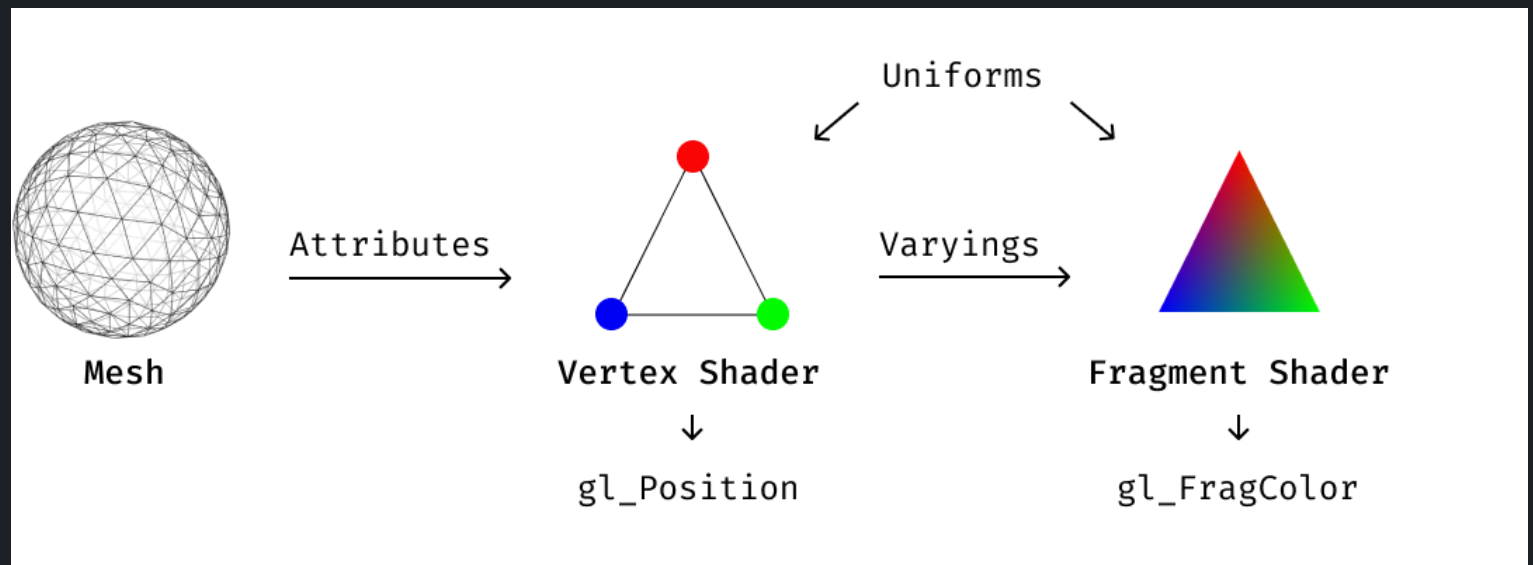
Runs on the GPU and controls how the graphics are rendered. Vertex shader is done on every vertex (positions), fragment is done on every pixel (color).

Application:

The vertex shader can change the shape of the geometry by manipulating every vertices' attributes. It can also change the color of the vertices itself (not the pixels in between the vertices, that is for the fragment shader), so only the corners of the polygons.

While fragment will help with defining colors, shading, and texture details. It can be used to produce gradient looks.

Diagram:



Examples:

Vertex: Used to manipulate meshes, so water/flag effect can have the vertex moving up and down using sine waves. Stretch and squash for bouncing motion? Make an object slimy or waterballoony. Grass and plants motion by swaying the vertices side to side.

Fragment: Color correction to emulate certain mood, lighting to enhance the scene (giving depth, or make it stylishly using toon/cel shading).

Glass and scrolling

What are glass and scrolling textures:

Glass gives transparency to the object, and can be controlled by a float value between 0 to one (0 is opaque, and 1 is invisible). Scrolling makes the texture move across an object, simulating movement.

Applications:

Glass can be used for glass and windows obviously, but it's for any object that can be transparent or even translucent (ie: water, jelly).

Scrolling is often used for water to simulate water flowing in a direction. This can also be used for anything that needs movement but the object itself does not move, for example: treadmills have movement on the tread, screens on a computer/tv any devices that displays moving images, maybe adjusted it so it moves like a slideshow)

Practical:

WATER SHADER

Shader Graphs

Water

Base texture

Texture2D

Base color

Color

Tiling

Vector2

Transparency

Float

Ripple color

Color

Ripple tiling

Vector2

Ripple speed

Float

Ripple density

Float

Ripple shininess

Float

Ripple edge sharpness

Float

Movement

Foam speed

Float

Wave tiling

Vector2

Wave speed

Float

Wave height

Float

Sea sway strength

Float

Graph Inspector

Node Set

Graph Set

Time

Animate stuff, in this case, its to move the uv for the wave height in a circle by offsetting the x and y value for offset. Using sine and cosine, allows me to move them back and forth, up and down, to create a circular motion

Noise

Makes the wave pattern more randomized as opposed to uniform. This makes the water based on real life, and also feels turbulent and violent to simulate the danger of it

Makes the base texture of the sea sways

Makes it more turbulent as well as creating depth when adding on the foam

Ripple/foam is based on noise

This creates randomization to the water ripple as well. This can also create more depth (illusion of more waves), which reduces amount of vertices transformed (less pointy and more huge wavy). So this basically makes the sea feels much powerful, large waves with smaller rippling waves within. Making it moves also creates depth when it move faster as well as in reverse because the foam is much closer to the viewer than the depth of the sea.

Reflection

Water is reflective, also makes the environment feels atmospheric and larger (mirror effect). It's also based on real water reflective property, which helps with immersion.

Main Preview

RIM SHADER

Shader Graphs

Rim color

Color

Rim power

Float

Rim glow

Float

World Space

Normal(3)

World Space

View Dir(3)

Rim power(1)

Power(1)

Fresnel Effect

Out(1)

Multiply

A(4)

B(4)

Out(4)

Multiply

A(4)

B(4)

Out(4)

Vertex

Object Space

Position(3)

Object Space

Normal(3)

Object Space

Tangent(3)

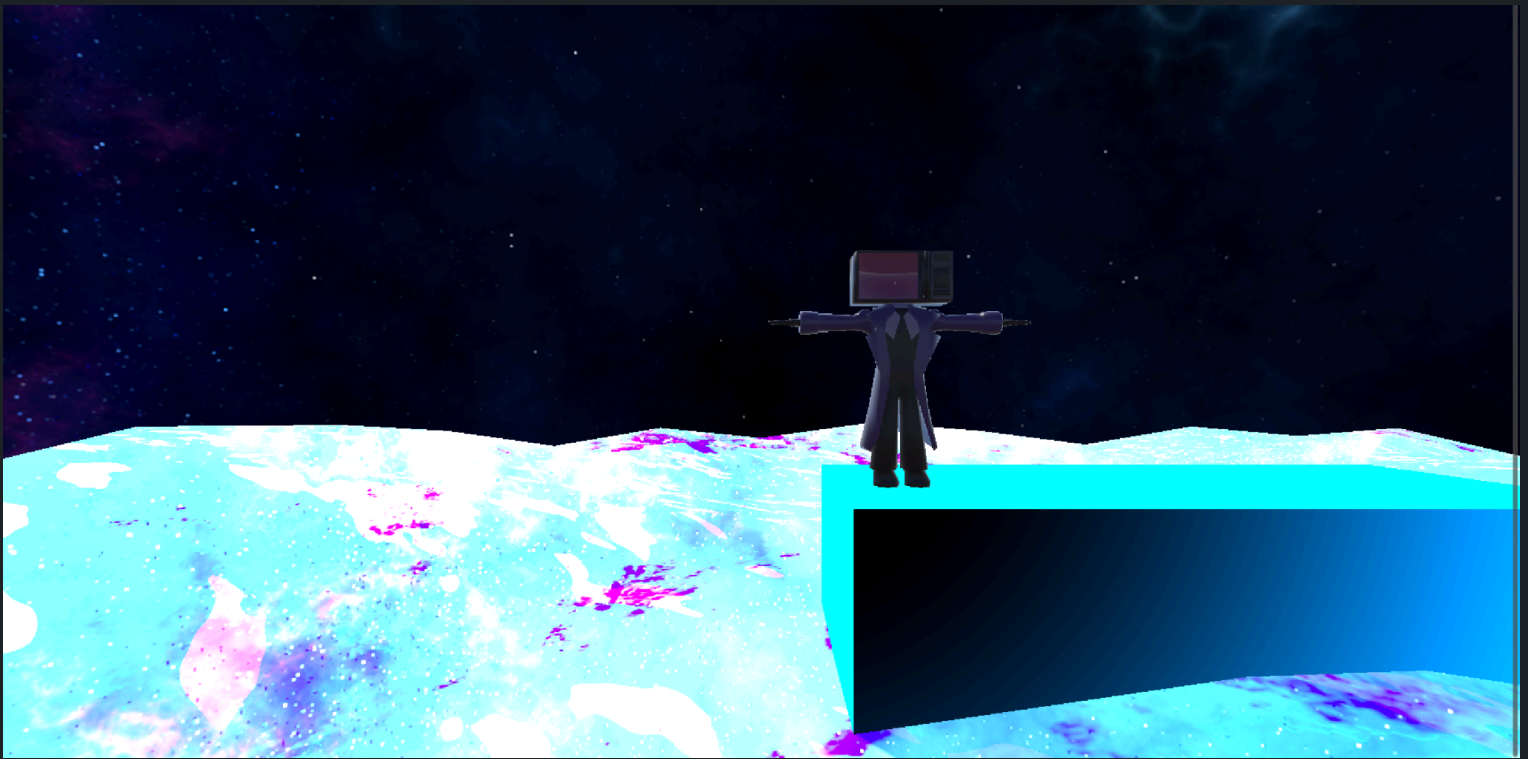
Fragment

Base Color(3)

I LOVE RIMMMM

cool glowy effect, idk why but im leaning toward a fantastical perspective, and the glow of the platform feels right as it seems like it reflected againts the bioluminescense of the sea of cosmic stars

Main Preview



Course evaluation:



Your responses were successfully submitted.

At Ontario Tech University, we strive to deliver a great education to each of our students in every one of our courses. There are many elements that contribute to improving the quality of a university course, including your perspective as a student. We listen to and value your input in helping us make your learning experience even better.

Please take the time to answer the following questions honestly, after careful reflection. Too often, student evaluations of courses are criticized for being popularity contests, focused on inappropriate aspects of their experience ("I think the prof dresses well"), or asks questions about things that students can't really evaluate ("did the instructor cover the field in this course sufficiently?"). Therefore we have tried to make this an evaluation of more substantive aspects of the course.

Once you've responded to the statements, please elaborate on any or all of your responses in the "Comments" section. These comments should be honest and direct but also respectful and constructive.

The instructor cannot see your name or other identifying information when they receive your feedback. However, abusive comments that are in violation of the [Student Conduct Policy](#) can be linked to your identity and may result in disciplinary action, and your feedback form will not be included in the course survey results shared with the instructor.

Thank you for helping us make Ontario Tech University courses even better by taking the time to complete this Student Course Feedback Survey!

Please select a course from the menu below:

Course not selected